

Secrecy Verification of Handshake Protocol using Squirrel Prover

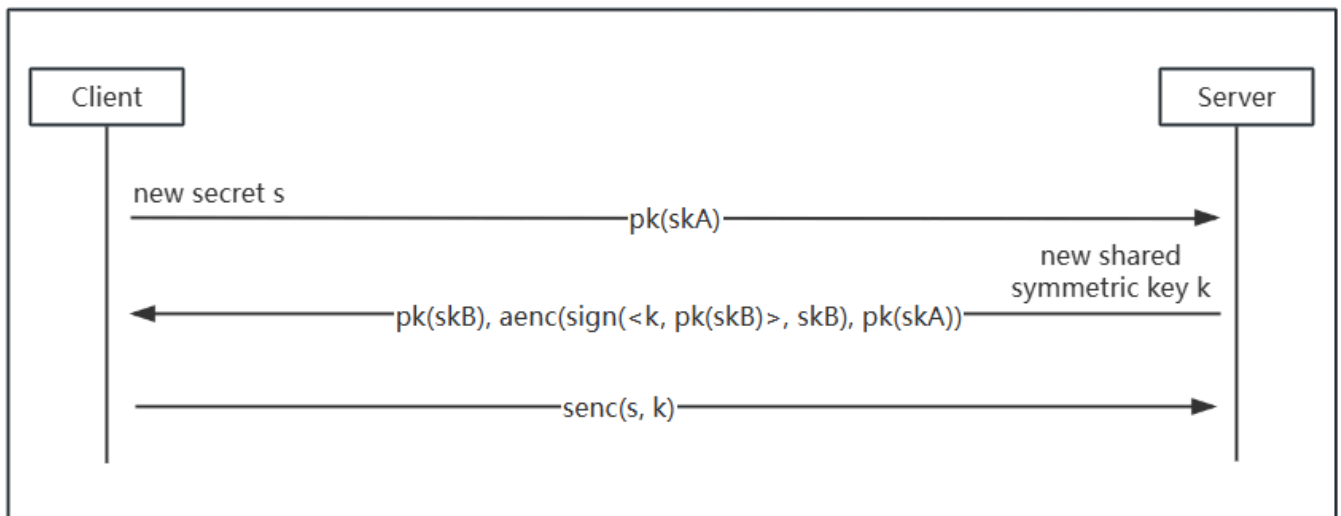
Table of Contents

- [Overview](#)
- [Project Objectives](#)
- [Strengths of Squirrel Prover](#)
- [Technologies - Logic & Formal Verification with Reachability and Equivalence properties](#)
- [Results](#)
- [Discussion and Future Work](#)
- [Academic Context](#)

Overview

This project presents a formal verification case study of a simplified handshake protocol and examines the usability of Squirrel Prover. It evaluates the practicality of applying Squirrel's existing framework to protocol verification in a post-quantum setting.

The simplified handshake protocol



Project Objectives

- Model a simple handshake protocol in Squirrel.
- Define secrecy properties for protocol participants.
- Construct a symbolic model and computational proofs of secrecy.
- Analyze proofs under the real and ideal projection systems.
- Document practical observations on proof automation, index management, auxiliary lemma development and proof scalability.

Strengths of Squirrel Prover

	CryptoVerif	EasyCrypt	Squirrel Prover
Modeling	Import predefined primitives Stateful protocols	Expressive -> Overhead↑ Properties of primitives	Stateful protocols Process algebra Interactive
Automation	Higher	Lower	Medium
Proof methodology	Indistinguishable oracles to adversary	Game-based proof framework	Frame & Traces Hard-coded(hypothesis)
Post-quantum support	Not support	qRHL & EasyPQC	PQ-BC

It is necessary to compare CryptoVerif, EasyCrypt, and Squirrel. Three of them aim to provide guarantees in the computational model.

CryptoVerif uses predefined primitives and supports stateful protocols. This makes it fast to use, but sometimes less flexible for new primitives. EasyCrypt is more detailed and expressive, which can introduce additional modeling overhead but makes it well suited to reasoning about properties of primitives. Unlike EasyCrypt, both CryptoVerif and Squirrel support verification of stateful protocols. The use of process algebra can be inconvenient for highly interactive protocols.

In terms of automation, Squirrel lies between the other two tools and is the first framework to provide completely mechanized protocol proofs using the CCSA approach. CryptoVerif uses indistinguishability-based oracle access to simulate adversary behavior, whereas EasyCrypt relies on a proof-oriented framework. Squirrel reasons about adversary knowledge and behavior through frames, traces and equivalence. Regarding hard-coded cryptographic hypotheses, verification often requires non-trivial syntactic checks and analysis of subterms.

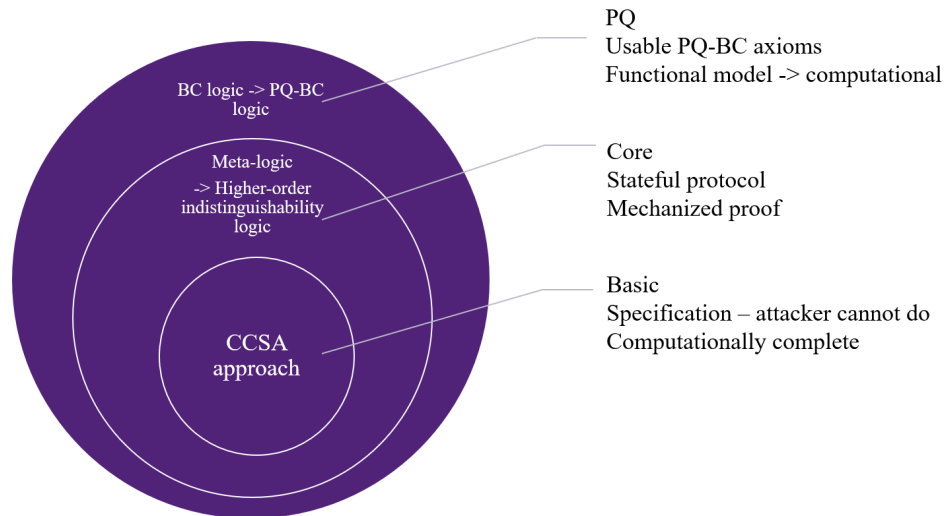
Finally, for post-quantum support, EasyCrypt and Squirrel can be combined with EasyPQC and PQ-BC style reasoning to adapt to the post-quantum setting. Squirrel has already been used to verify a wide range of protocols, including properties such as authentication, unlinkability, secrecy and anonymity. An important trend is that Squirrel not only supports symbolic modeling but also extends to computational and post-quantum reasoning.

Overall, Squirrel Prover makes a significant step toward bridging symbolic verification techniques and computational cryptography within a single proof framework, thereby adapting protocol verification to the post-quantum setting.

- Expressive modeling language for symbolic protocols.
- Support for equivalence and Reachability in proof.
- Interactive verification through tactics and trace.

- Strong support for post-quantum computational soundness.

Technologies - Logic & Formal Verification with Reachability and Equivalence properties



CCSA is the basic or underlying logic. It relies on specifying what the adversary cannot do and provides computational soundness for the symbolic model. The framework is based on a meta-logic and can be extended to higher-order indistinguishability reasoning. This allows Squirrel to reason about stateful protocols and support mechanized proofs. The extended PQ-BC logic adapts Squirrel to the post-quantum setting and defines usable PQ-BC axioms from cryptographic assumptions.

Declaration	Protocol process	Reachability in proof	Equivalence in proof
Define secrets, keys, channel, functions	Model the commutations between sender and receiver	Unreachable	Indistinguishability
Name(index) w/o assumption Basic assumption Cryptographic assumption	Trace	Trace Macros Timestamp & Actions	Equivalence formula - <i>equiv</i> (_,_)
/	Symbolic Computational CCSA	Security properties – <i>diff</i> (<i>real</i> , <i>ideal</i>) -> Real/ideal system	

Logic support Squirrel to structure verification work, and there are four components during verification. Declaration define some messages and different kinds of function symbols we will use in modelling.

The Protocol process models the communication behavior between protocol agents and then records execution traces for further analyze.

The Squirrel Prover also involves reachability and equivalence properties. Reachability properties reason about whether an adversary can obtain a target value in any protocol trace. They often use timestamps, actions, and traces to describe protocol executions. For example, the weak secrecy property proves that the adversary cannot derive the secret from the frame `frame@t`. The `frame@t` contains all outputs observable before and up to time `t`, representing the knowledge available to the adversary at that point.

Equivalence properties more focus on the equivalence between a real system and an ideal system. The real and ideal systems are projections of the Protocol. To be more specific, the real system models the actual protocol, while the ideal system represents a perfectly secure version of the protocol. Instead of proving directly that an adversary cannot obtain a secret, Squirrel Prover proves that the adversary cannot distinguish interactions with the real system from interactions with the ideal system. If the two systems are indistinguishable, the desired security property follows under the corresponding computational assumptions.

Results

The verification demonstrates that the modeled protocol satisfies the specified secrecy property under the symbolic adversary model. After `set postQuantumSound = true`, the secrecy property of this case study remained valid. This switch provides straightforward between classical and post-quantum environment in Squirrel Prover. However, the attempt at verification of secrecy under the real/ideal system was unsuccessful. The complexity of equivalence reasoning in this setting was increases. In addition, induction over protocol traces and index management challenges may have obstructed the completion of the verification. These results both demonstrate the capabilities of Squirrel Prover in computational complete and the current challenges under real/ideal system.

Discussion and Future Work

During the verification of the real/ideal system proof, there are some limitations in automated tactics, such as actions became increasingly complex. In addition, error messages generated by automated tactics were often insufficiently detailed and difficult to pinpoint the source of failure. Automated tactics, such as `auto`. In this case study, the verification returned false when attempting to resolve the name and index matching problem. while debugging process, the message also failed to clearly indicate the main reason. These findings indicate that improving the usability of proof development could be future research. Researchers could also evaluate Squirrel Prover on more complex protocols and security properties, such as TLS protocol and forward secrecy property.

Academic Context

This project was a capstone project in my last year of master's degree(July 2025).

Squirrel Prover Reference: <https://squirrel-prover.github.io/>